

Discente: Wilton Terto Leal / Matrícula: 20240011543

## **Lista de Exercícios - Arranjos, Strings, Estruturas e Multi-arquivo**

### **Questões Teóricas**

1. Os arrays em C começam no índice 0. Um array de tamanho N vai da posição 0 até a posição N-1. A condição  $i \leq N$  faria o laço tentar acessar o índice N, que está fora do bloco de memória alocado, causando comportamento indefinido.
2. Em C, o nome do array atua como um ponteiro para o primeiro elemento. A notação  $v[i]$  é apenas um atalho sintático (syntactic sugar) que o compilador traduz automaticamente para a operação de aritmética de ponteiros  $*(v + i)$ .
3.
  - (a) O operador sizeof calcula e retorna o tamanho, em bytes, que uma variável ou tipo de dado ocupa na memória.
  - (b) size\_t é um tipo de dado inteiro padronizado retornado pelo sizeof, usado para representar de forma segura o tamanho e a contagem de objetos em memória.
  - (c) Ele é um inteiro sem sinal (unsigned) porque o tamanho de um objeto na memória jamais pode ser um valor negativo, permitindo assim armazenar valores maiores do que os tipos inteiros com sinal.
4. Você calcula o número de elementos dividindo o tamanho total ocupado pelo array inteiro na memória (sizeof(ar)) pelo tamanho que apenas o primeiro elemento ocupa (sizeof(ar[0])).
5. Quando você passa um array como parâmetro, ele "decai" para um ponteiro comum apontando para o seu primeiro elemento. O sizeof(ar) retornará o tamanho do ponteiro na arquitetura do sistema (geralmente 4 ou 8 bytes), e não o tamanho do array original.
6. Na aritmética de ponteiros, a linguagem C multiplica o número somado pelo tamanho do tipo de dado que o ponteiro referencia.
7. A expressão  $p + 3$  soma **3 posições** à frente no array. Se o ponteiro for de int (tipicamente 4 bytes), a soma no endereço físico da memória seria de 12 bytes ( $3 \times 4$  bytes), mas logicamente ele aponta para o 4º elemento.
8.
  - (a)  $p + 4$  representa o endereço de memória do quinto elemento do array (índice 4).
  - (b)  $*(p + 4)$  desreferencia esse endereço, representando o valor armazenado no quinto elemento.
  - (c)  $p_2 - p_1$  representa a distância em memória, em termos de "número de elementos", entre os dois ponteiros.

**9.** A variável `p` recebe o endereço do início do array. Portanto, acessar a memória deslocando o ponteiro `p` com `*(p+i)`, usar a notação de colchetes `ar[i]`, ou deslocar diretamente o endereço base `*(ar+i)` resultam exatamente na mesma posição de memória e no mesmo valor.

**10.** Em termos de processamento, não há diferença; ambas instruem o compilador que a função espera receber um endereço de memória contendo valores do tipo `float`. A notação `float ar[]` serve apenas como convenção visual para indicar ao programador que o ponteiro será tratado como uma coleção sequencial.

**11.** Como o array decai para ponteiro na chamada da função perdendo sua informação de tamanho, a função não tem como saber onde a coleção termina. O parâmetro de tamanho evita acessos inválidos de memória.

**12.** Arrays em C não são variáveis regulares que carregam todo seu conteúdo na atribuição; seus identificadores são lidos apenas como endereços. Quando a função é encerrada, os dados na pilha local são destruídos, impedindo o retorno do bloco de forma segura por valor.

**13.**

**(a)** Uma variável zumbi é uma variável local que foi destruída pelo fim da execução de sua função, mas ainda tem seu endereço de memória referenciado em alguma parte do código.

**(b)** Incorreto porque tenta retornar um array declarado localmente.

**(c)** O ponteiro retornado aponta para um local da pilha que agora está livre para ser reescrito pelo sistema, trazendo dados inválidos e comportamento imprevisível.

**(d)** Você deveria alocar o array com `malloc` (que sobrevive ao encerramento da função) ou fazer o chamador da função enviar um ponteiro pré-alocado como parâmetro.

**14.**

**(a)** Um bloco de memória é uma sequência contínua de bytes disponibilizados na RAM para uso pelo programa.

**(b)** A função `malloc()` reserva a quantidade solicitada de bytes no *Heap* (área de memória dinâmica) e retorna o endereço inicial do bloco.

**(c)** Indica que a alocação falhou, normalmente porque o sistema operacional ficou sem memória RAM disponível.

**(d)** Previne a criação de "dangling pointers" (ponteiros selvagens). Isso evita que o programa modifique a memória acidentalmente num endereço já devolvido.

**15.** O tamanho dos tipos primitivos pode variar dependendo da arquitetura da máquina (um `int` pode ter 2 bytes em sistemas antigos e 4 bytes em sistemas modernos). O `sizeof(tipo)` assegura que a quantidade correta seja reservada para qualquer sistema operacional.

**16.** A função `free()` devolve a área dinâmica para o sistema, prevenindo vazamentos de memória (memory leaks) que podem causar o travamento do programa. Fazer `p = NULL` em seguida corta o vínculo do programa com o espaço descartado.

**17.** Na aritmética de matrizes (arrays aninhados), o compilador necessita das dimensões secundárias para calcular quantos bytes pular até chegar na próxima "linha". Sem esses tamanhos definidos, a fórmula de deslocamento quebra.

**18.** Aloca-se primeiro um array de ponteiros representando as "linhas". Em um laço de repetição, para cada linha do array de ponteiros, executa-se um novo malloc() criando as matrizes isoladas que vão agir como colunas.

**19.** char \*p = "abc" aponta para uma string literal criada numa área de memória "somente leitura" (read-only). char s[] = "abc" inicializa um array local na pilha do programa, copiando a literal para lá. Apenas a forma de vetor de array (s[]) pode ter o conteúdo modificado de forma segura.

**20.** \* strlen: Percorre a string para contar os caracteres lógicos antes de encontrar o terminador \0.

- strcmp: Compara duas strings caractere por caractere (alfabeticamente), retornando 0 se forem iguais.
- strcpy: Copia sequencialmente o conteúdo de uma string para dentro de um bloco de memória alocado de outra.
- Não devemos usar == porque este operador comparará apenas os endereços onde as strings estão guardadas na memória, não o texto contido nelas.

**21.**

**(a)** Arquivos .h armazenam interfaces (declarações), e .c armazenam as implementações da lógica.

**(b)** Protótipos de funções, structs, definições de typedef, macros, e diretivas de inclusão globais.

**(c)** Isso duplica a implementação durante o estágio de *linkagem*, gerando falha fatal de "múltiplas definições da mesma função" no compilador.

**22.**

**Escopo:** *De programa:* Visível em todo o projeto. *De arquivo:* Visível apenas dentro do arquivo fonte corrente. *De bloco:* Restrito ao bloco englobado pelas chaves {} onde foi criado.

**Duração:** *Automática:* A memória é liberada assim que sai do seu escopo local. *Fixa (static):* A memória perdura intocada do início ao fim da execução do programa.

**23.**

**(a)** Limita a variável global para que apenas funções do arquivo atual consigam acessá-la, bloqueando acessos via diretiva extern de outros arquivos.

**(b)** Faz a variável sobreviver às sucessivas execuções da função, não perdendo seu valor retido nas chamadas anteriores.

**(c)** Isola e encapsula a função para uso apenas pelo seu próprio arquivo (módulo), prevenindo conflitos com funções de mesmo nome no projeto.

**24.**

**(a)** É um tipo de dado composto projetado pelo usuário capaz de agrupar sob uma mesma tag dados de diferentes tipos lógicos relacionados.

**(b)** O operador . puxa o membro de uma estrutura normal instanciada, enquanto -> desreferencia a posição apontada e puxa o membro por intermédio de um ponteiro.

**(c)** Previne uma cópia integral do bloco de dados via pilha de execução, ganhando extrema eficiência e menor uso de memória.

25. Você o declara através de um array convencional `struct Aluno turma[50]`; e o acessa cruzando índice e ponto, ex: `turma[0].nota`. Elas são descritas como heterogêneas porque um único índice permite o agrupamento de `int`, `float`, e vetores sob a mesma abstração.

26.

(a/b) Na `struct`, a memória demandada corresponde à soma dos bytes de todos os membros, onde cada um possui área individual. Na `union`, um único bloco do tamanho de seu membro mais pesado hospeda temporariamente um dado por vez.

(c) Em protocolos de rede ou sistemas com memória muito restrita que só preenchem um tipo de dado a cada operação lógica exigida.

27. O `enum` fornece checagem de tipos aprimorada pelo compilador, permite que o depurador exiba nomes textuais claros ao invés de valores em cru, e facilita muito a leitura lógica e a manutenção das opções disponíveis num sistema de estados.

28. A primeira `tRegistro func()`; executa uma cópia exata de todos os bytes internos da `struct`, preenchendo o barramento de memória desnecessariamente. A segunda notação por ponteiro `tRegistro* func()`; é infinitamente mais **eficiente**, pois transfere apenas um endereço (geralmente 8 bytes). O ponteiro retornado deve imperativamente referenciar um objeto criado via `malloc` (dinamicamente) ou pré-alocado, do contrário, produzirá uma variável zumbi inútil.